Can I trust this model? Overfitting, cross-validation and leakage

How do we know a model that predicts cooling load will still work on a building it has never seen?

CML3112 | Civil Engineering and Architecture | Week 10, class 3 of 4 | Tuesday 17 November 2026, 11:00 to 14:00

KLAi | International University of East Africa | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Explain underfitting, a good fit and overfitting (the bias-variance trade-off), and tune the hyperparameter max_depth with a validation curve



Work out training error, validation error, the generalisation gap, and a cross-validation mean and standard deviation, by hand



Score a model with K-fold and GroupKFold cross-validation, and report the mean with its standard deviation



Name three kinds of data leakage and use a checklist that blocks them

Your three hours: ideas first, then the lab

0:00	10	Recall from Class 2, and why trust matters
0:10	40	Concepts, part 1: overfitting, validation curve, cross-validation
0:50	30	Concepts, part 2: data leakage, random seed, pipeline
1:20	15	Break
1:35	75	Lab: validation curve, cross-validation, leakage, seeds
2:50	10	Close and how this class counts

Concepts first, with no laptops. Then the lab, with the notebook open.

Three questions from last time

1. Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?

2. In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?

3. What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?



Answer aloud first. Then we check together.

Answers to the three questions from Class 2

Question	Short answer
Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?	It finds none of the real positive cases, so its recall is 0. Accuracy hides that.
In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?	False negative: the model said no (negative) and the truth was yes (positive). False positive: the model said yes and the truth was no.
What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?	Recall rose from 0.67 to 1.00. False positives went from 0 to 1.

Words we use today

	Word	What it means
	**Overfitting**	The model fits the noise in the training set (it nearly memorises it) and does worse on new rows. Underfitting is the opposite: too simple to learn the pattern.
	**Cross-validation**	Scoring a model K times. The rows are cut into K parts called folds. Each fold takes one turn as the validation fold (held-out fold).
	**Data leakage**	Information the model would not have at prediction time (held-out rows, or the target itself) reaches it while it is trained, so the score looks better than it really is.
	**Bias-variance trade-off**	Too simple a model misses the pattern (high bias, underfitting). Too flexible a model chases noise (high variance, overfitting).
	**Random seed**	A fixed number (random_state in scikit-learn) that makes a random choice repeat exactly.

A model that looks good on paper can fail on a new machine or site



A model is built from data on four buildings and then used on a fifth building.

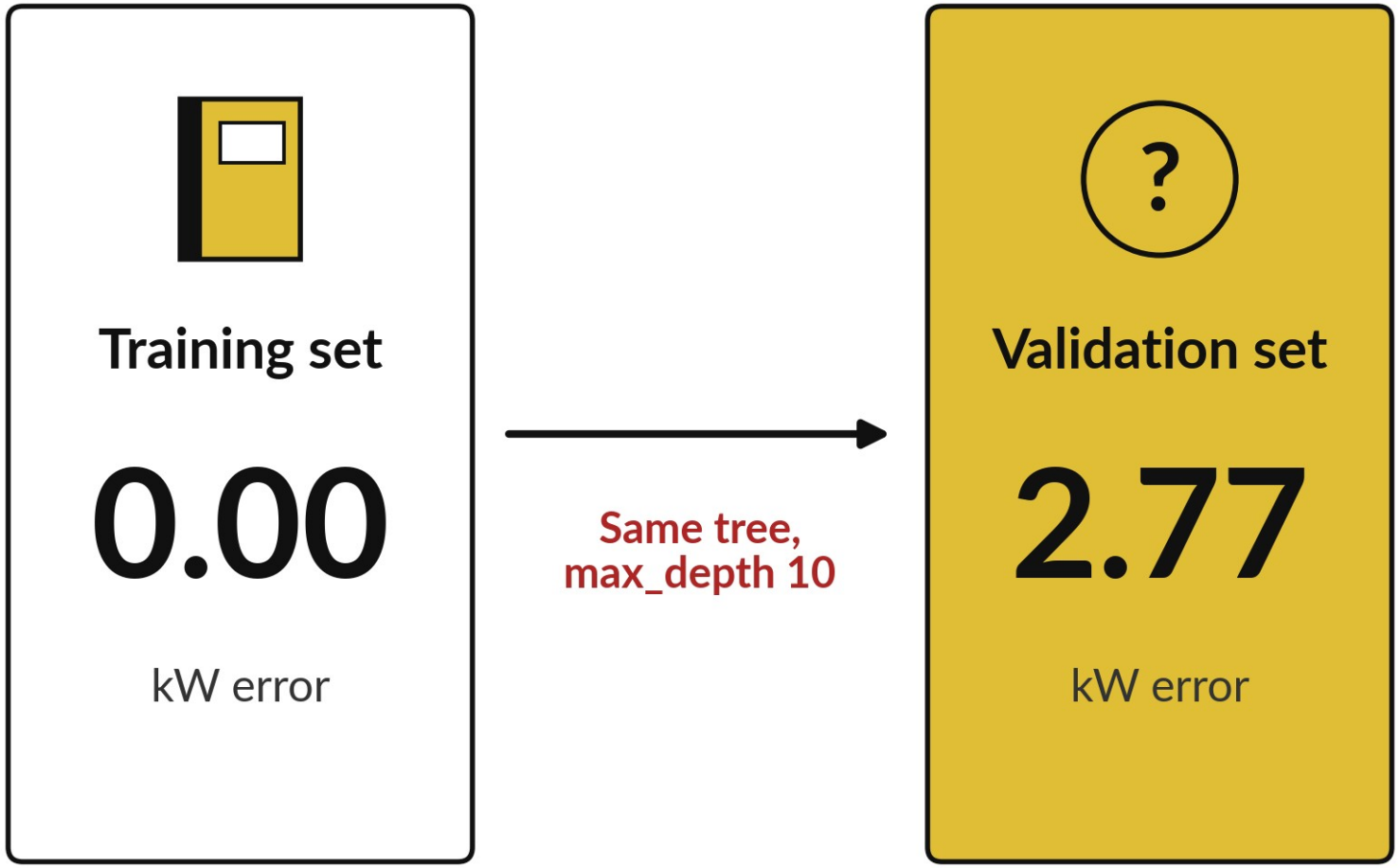


The size of the cooling plant is chosen from the model's prediction.



If the prediction is wrong, the building is too hot or the plant is too big.

A model must do well on rows it has never seen

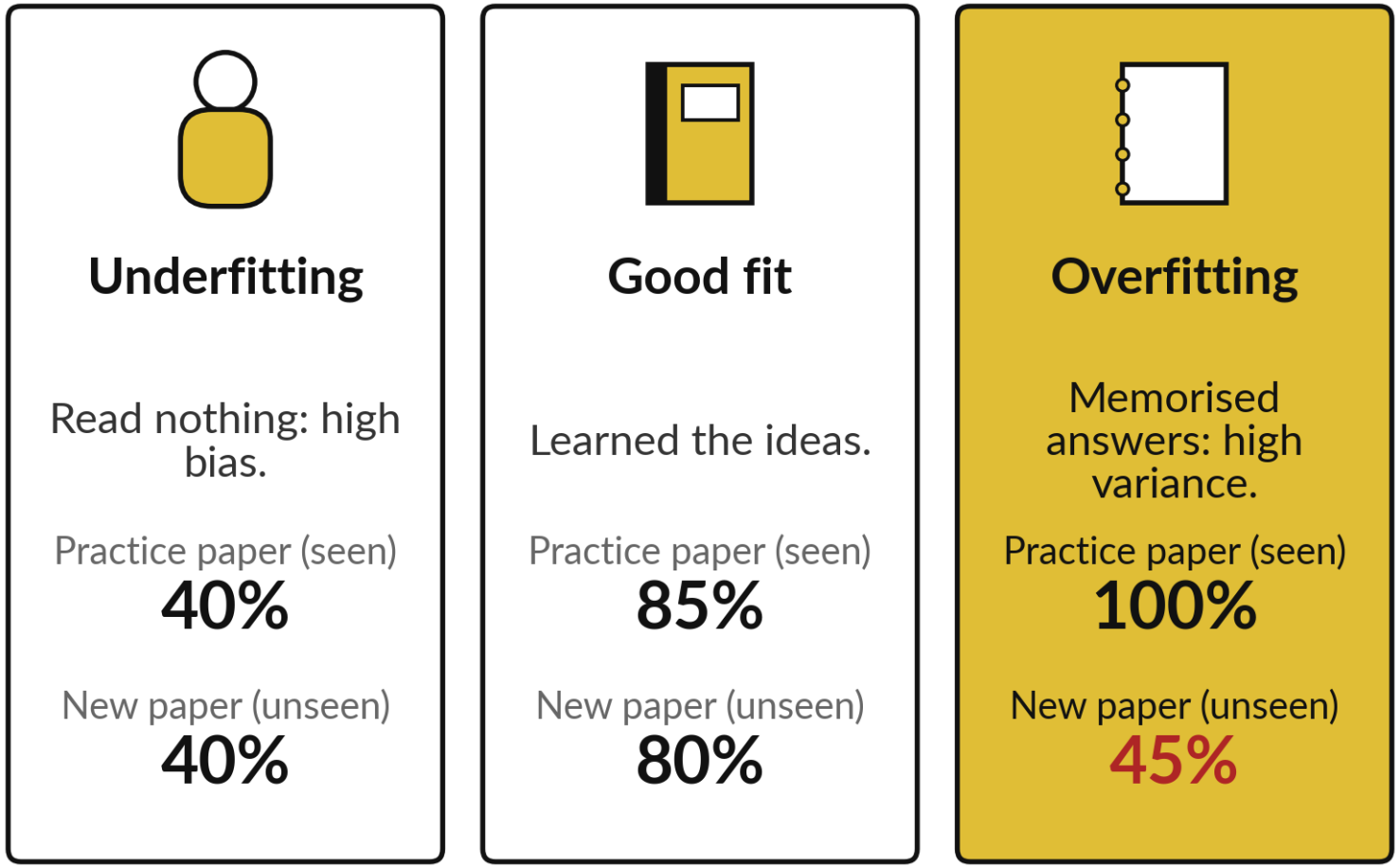


Generalisation means doing well on held-out data: rows the model has never seen.

The tree was trained on the training set, so its error there is almost zero.

The validation set was held out. On it the prediction error is far larger. The difference is the generalisation gap.

Underfitting, a good fit and overfitting look like three students



The marks in the picture are illustrative (made up). Each student stands for one kind of model.

Practice paper = training set. New paper = held-out rows.

Bias-variance trade-off: too simple a model misses the pattern, too flexible a model chases noise.

Reminder from Class 1: MAE and RMSE measure the prediction error

MAE: mean absolute error

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

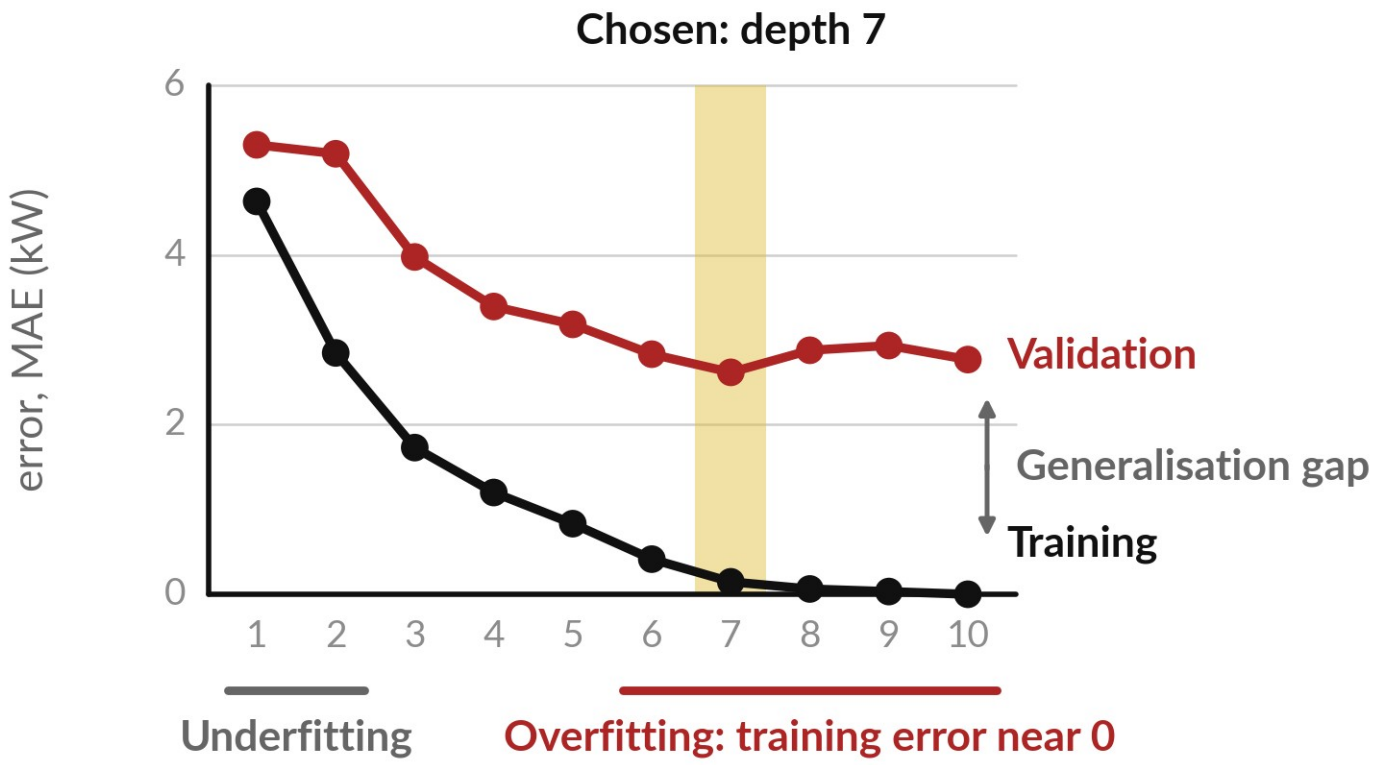
MSE and RMSE

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

Symbol	What it means	In our data
MAE	The mean size of the error. The bars drop any minus sign	Class 1 model: 0.74 kW
MSE, RMSE	MSE: mean of the squared errors. RMSE: its square root, in the target unit	Class 1 model: RMSE 0.88 kW
n, i, y, ŷ	n rows scored; i is the row number; y_i is the actual target of row i; ŷ_i (y-hat) is its prediction	15 validation rows; cooling load in kW

Illustrative example (made up): errors of 1, 2 and 3 give MAE = 2, MSE = 4.67 and RMSE = 2.16.

The validation curve: as max_depth grows, training error keeps falling but validation error stops improving



A validation curve plots both errors against max_depth, a hyperparameter (set before training).

Each dot is one tree. Black: training error. Red: validation error. Between them: the gap.

Depths 1 and 2: both errors high. From depth 6: training error near 0. Gold: chosen depth 7.

Training error, validation error and the generalisation gap are short formulas

Training error

$$\mathrm{MAE}_{\mathrm{train}} = \frac{1}{n_{tr}} \sum_{i \in \mathrm{train}} |y_i - \hat{y}_i|$$

Validation error

$$\mathrm{MAE}_{\mathrm{val}} = \frac{1}{n_{va}} \sum_{i \in \mathrm{val}} |y_i - \hat{y}_i|$$

Symbol	What it means	In our data
MAE	Mean absolute error: the average size of the prediction error. The small label train or val says which rows	Depth 10: 0.00 and 2.77 kW
n	How many rows. The small label tr means training and va means validation	58 and 15 rows
i	A row number. The sum adds up the rows of that group	Row 1, row 2, and so on
y, ŷ	The real target of row i, and the model's prediction for it. The bars | | drop any minus sign	cooling load and the tree's answer, in kW

The generalisation gap is the validation error minus the training error

Generalisation gap

$$\text{gap} = \text{MAE}_{\text{val}} - \text{MAE}_{\text{train}}$$

Depth 10

$$\text{gap} = 2.77 - 0.00 = 2.77 \text{ kW}$$

Depth 7

$$\text{gap} = 2.62 - 0.15 = 2.47 \text{ kW}$$

Symbol	What it means	In our data
gap	Generalisation gap: how much worse the model is on new rows	Depth 10: 2.77 kW
MAE_val	Validation error, from the formula before	2.77 kW at depth 10
MAE_train	Training error, from the formula before	0.00 kW at depth 10

Small gap: similar error on new rows. Big gap: overfitting. We choose max_depth by validation error.

Five depths in one table: leaf nodes, errors and the generalisation gap

max_depth	Leaf nodes	Training MAE	Validation MAE	Generalisation gap (validation minus training)
1	2	4.63	5.30	0.67
2	4	2.84	5.19	2.35
4	15	1.20	3.39	2.19
7 (chosen)	46	0.15	2.62	2.47
10	58	0.00	2.77	2.77

MAE in kW. A leaf node is a final node: it holds the prediction. The Class 1 linear regression model scores 0.74 kW, better than every tree: the target is close to a straight line, and a tree only makes steps.

We choose max_depth on the validation set and keep the test set sealed



The tree is trained here.

We compare depths here.

Sealed. Opened once, at the end.

Each set is made of whole equipment groups. The picture shows the rows in each set.

Only 15 validation rows, so a score can move by luck. Never tune on the test set.

Test set rule: open it once, after the model and any threshold are chosen. Refit the chosen model on E1 to E5, score it once on E6.

Tuning a hyperparameter: choose the simplest max_depth within 5 percent of the lowest validation error

Tolerance limit

$$\text{limit} = 1.05 \times V_{\min}$$

Chosen depth

$$d^* = \text{smallest } d \text{ with } V(d) \leq \text{limit}$$

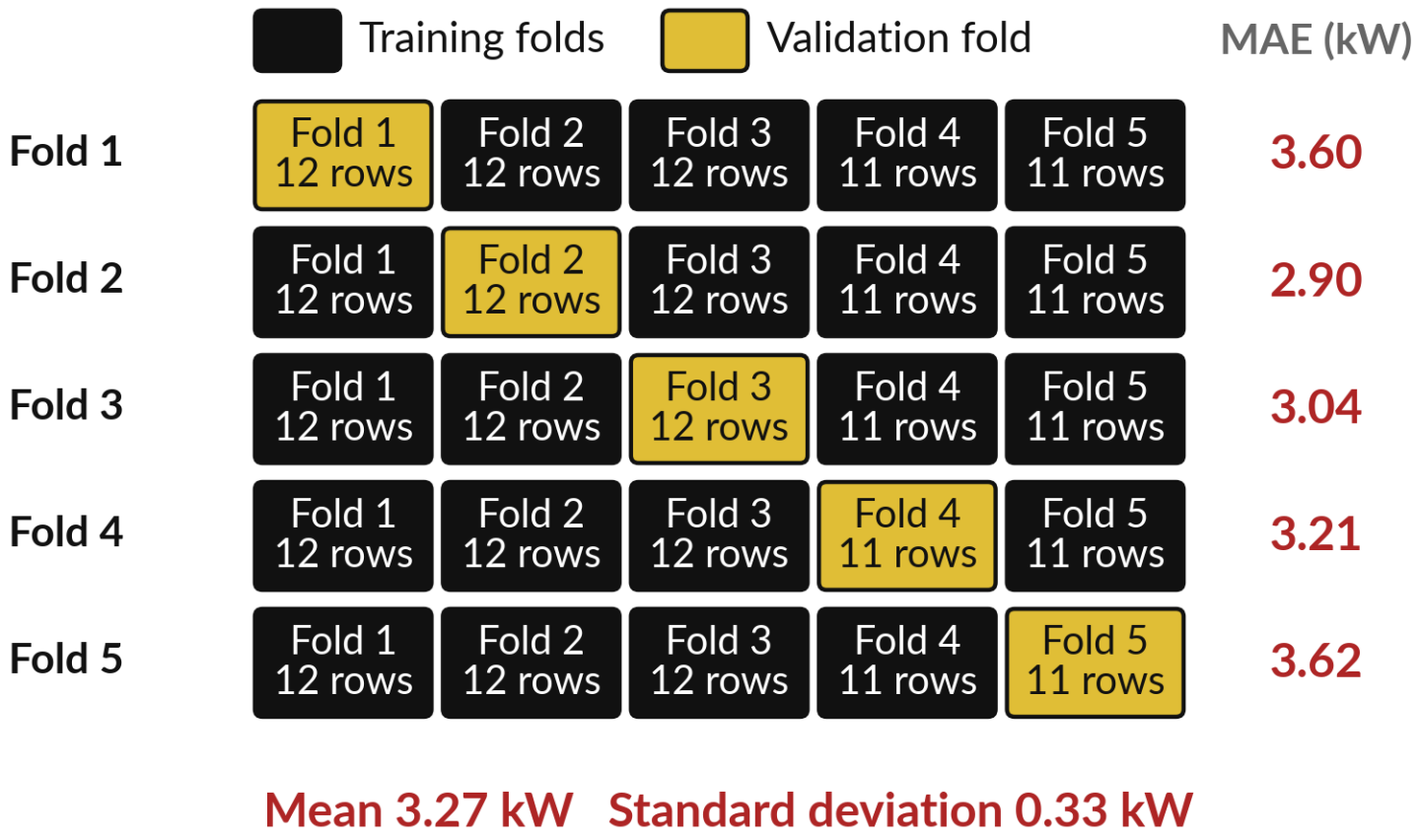
Our data

$$1.05 \times 2.616 = 2.747 \text{ kW}, \; d^* = 7$$

Symbol	What it means	In our data
V(d)	Validation MAE of the tree with max_depth d	5.30 at depth 1, 2.77 at depth 10
V_min	The lowest validation MAE over depths 1 to 10	2.616 kW, at depth 7
limit	The tolerance limit: the lowest value plus 5 percent	2.747 kW
d*	The chosen depth: the smallest d that passes	Depths that pass: 7

Only depth 7 passes: here the rule and the lowest score agree. d* = chosen max_depth (parsimony).

One split can be lucky, so K-fold cross-validation lets every fold take a turn as the validation fold



K-fold cross-validation cuts the 58 training rows into K = 5 folds (parts of the rows), in the order the rows appear (no shuffle).

In each line, the tree is trained on 4 folds and scored on the gold validation fold.

Rows of E1 to E4 are interleaved: every fold holds all four groups (fold 1: E1 4, E2 4, E3 2, E4 2).

K-fold gives k scores, and we report their mean and their standard deviation

Mean of the k scores

$$\bar{s} = \frac{1}{k} \sum_{j=1}^{k} s_j$$

Sample standard deviation of the k scores

$$\sigma_s = \sqrt{\frac{1}{k-1} \sum_{j=1}^{k} (s_j - \bar{s})^2}$$

Symbol	What it means	In our data
s_j	The MAE when fold j is the validation fold (j is the fold number)	3.60, 2.90, 3.04, 3.21, 3.62 kW
k	The number of folds, so the number of scores	5 folds
s̄	The mean (average) of the k scores (s with a bar)	3.27 kW
σ_s	Sample standard deviation: typical distance of a score from the mean. np.std(scores, ddof=1)	0.33 kW

By hand with four illustrative scores: the mean is 3 and the standard deviation is 0.82

Fold	Score (MAE)	Distance from the mean	Squared distance
1	2	-1	1
2	3	0	0
3	3	0	0
4	4	+1	1

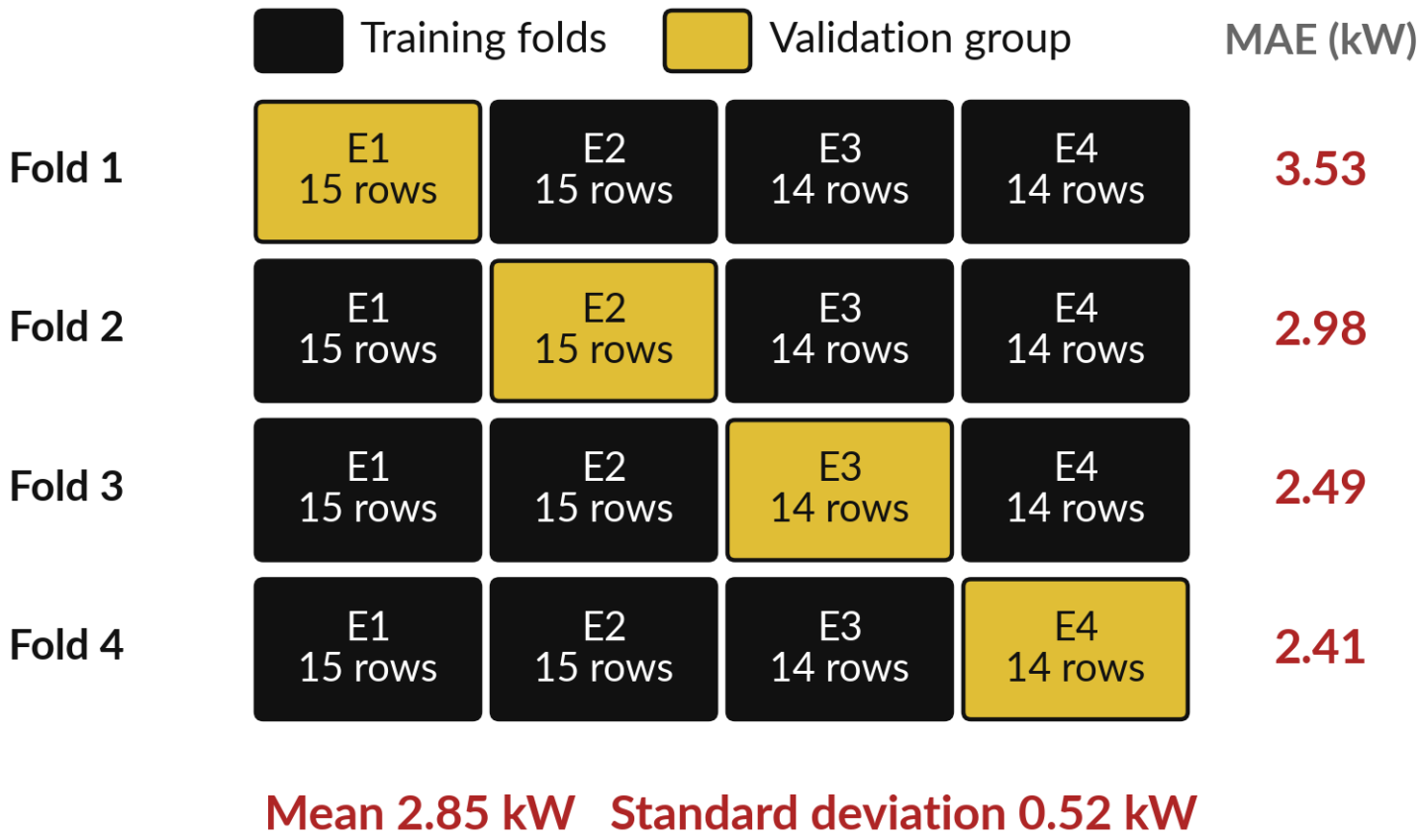
Mean = (2 + 3 + 3 + 4) / 4 = 3. Sum of the squared distances = 2. Divide by k - 1 = 3: 2 / 3 = 0.67 (the sample variance). The square root of 0.67 is 0.82, the standard deviation. These four scores are illustrative (made up).

The same steps on our five real fold scores give their mean and standard deviation

Fold	Score (MAE, kW)	Distance from the mean	Squared distance
1	3.60	+0.32	0.104
2	2.90	-0.38	0.143
3	3.04	-0.23	0.054
4	3.21	-0.06	0.004
5	3.62	+0.35	0.122
Sum	16.37	0.00	0.427

Mean = 16.37 / 5 = 3.27 kW. Sum of squares / (k - 1) = 0.427 / 4 = 0.107. Square root = 0.33 kW, the standard deviation. Worked from unrounded scores, so the last digit can differ.

GroupKFold holds out a whole equipment group in each fold, which is how the model is really used



GroupKFold keeps each group whole: each of E1 to E4 is held out once (scikit-learn picks the order).

Plain K-fold put every building in every fold. Here the tree trains on three groups and is scored on the fourth, like a new site.

One score per fold; mean and standard deviation as before.

GroupKFold uses the same formulas, with equipment groups as the folds

Score of group g

$$s_g = \frac{1}{n_g} \sum_{i \in g} |y_i - \hat{y}_i|$$

Mean and standard deviation, 4 groups

$$\bar{s} = \frac{1}{4} \sum_{g} s_g, \quad \sigma_s = \sqrt{\frac{1}{3} \sum_{g} (s_g - \bar{s})^2}$$

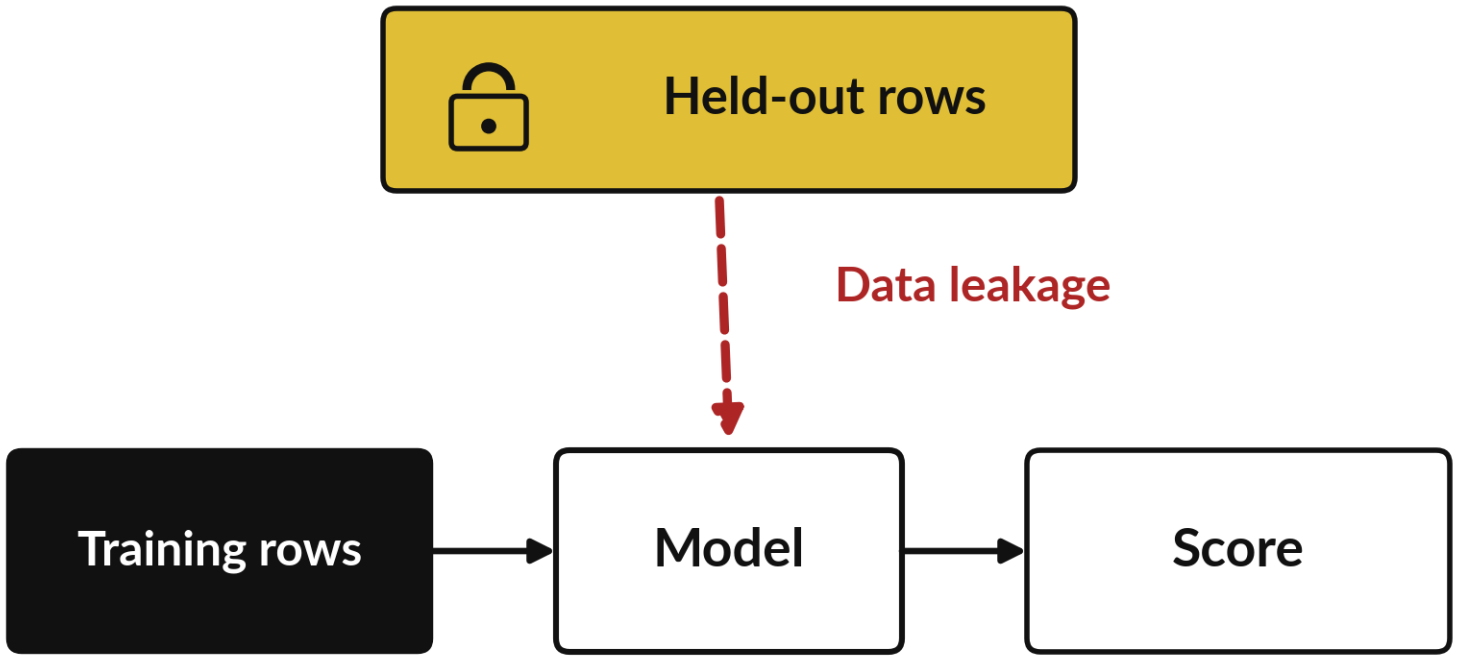
Symbol	What it means	In our data
g	The group held out in one fold: E1, E2, E3 or E4	E1 to E4, one per fold
n_g	Number of rows in group g	E1 15, E2 15, E3 14, E4 14 rows
s_g	MAE on group g, using a tree that was never trained on g	E1 3.53, E2 2.98, E3 2.49, E4 2.41 kW
s̄, σ_s	Mean of the 4 scores; sample standard deviation (divide by 3)	2.85 and 0.52 kW

The same max_depth 4 tree gets three different scores, depending on how we test it

	Way of testing	Scores (MAE)	Mean	Standard deviation
	**Single validation split (group E5)**	3.39	3.39	one score only
	**K-fold, k = 5 (rows in order, no shuffle)**	2.90 to 3.62	3.27	0.33
	**GroupKFold, k = 4**	2.41 to 3.53	2.85	0.52

MAE in kW. The single E5 score is within about one standard deviation of both means: the normal spread from one fold to another. The standard deviation shows how far to trust each mean.

Data leakage is information the model would not have at prediction time reaching it during training



Normal path: training rows train the model, then it gets a score

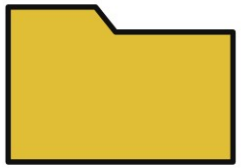
Held-out rows must never reach the model

Data leakage: information the model would not have at prediction time (held-out rows, or the target itself) reaches it while it is trained.

Held-out rows are the validation or test rows. The dashed red arrow is the leakage.

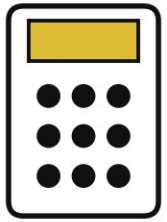
The score looks excellent, then the model fails on real new rows.

Data leakage comes in three common kinds



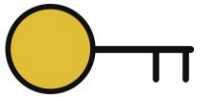
1 Train-test contamination

Validation rows are in the training set



2 Preprocessing leakage

Imputer (fills missing values) or scaler (rescales columns) fitted on all rows

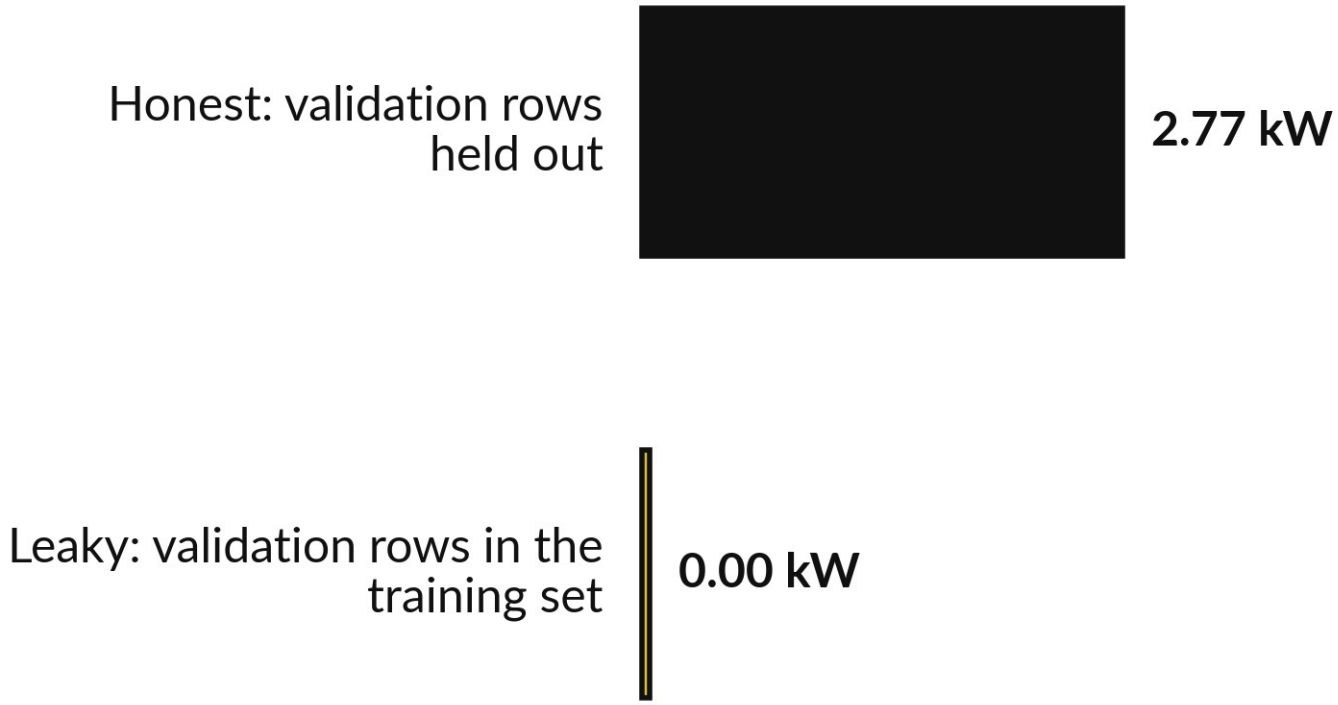


3 Target leakage

A feature is built from the target

All three make the score look too good.

Train-test contamination: validation rows in the training set make the score look perfect

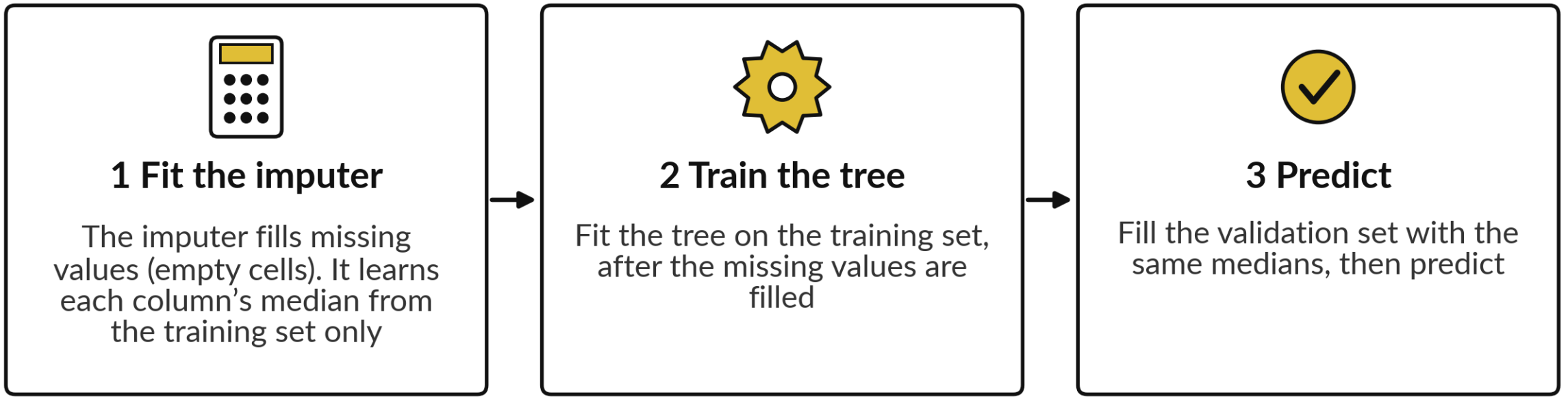


Same max_depth 10 tree. Only the training set changed.

In the leaky run the tree was already trained on the validation rows, so it reproduces their answers: it has already seen them.

The honest score is the real one.

A Pipeline fits the imputer on the training set only, which prevents preprocessing leakage



Median of ambient_c: 26.07 from the training set, 25.37 from training plus validation rows (73 rows, leakage). Only one value is missing, so the score did not change.

The pipeline rule: fit on the training set only, then apply the same numbers

Imputer: the median fill

$$m_j = \text{median}\{\, x_{ij} : i \in \text{train} \,\}$$

StandardScaler: mean, then z-score

$$\mu_j = \frac{1}{n_{tr}} \sum_{i \in \text{train}} x_{ij}, \qquad z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

Symbol	What it means	In our data
i, j, x_ij, n_tr	Row i, column j, the value x_ij there, n_tr training rows. In preprocessing leakage, i also runs over validation rows	j = 1 load_pct, j = 2 ambient_c; n = 58
m_j	Median of column j over the training rows	ambient_c: 26.07. Training plus validation: 25.37 (leakage)
μ_j, σ_j	Mean and standard deviation of column j, training rows (divide by n_tr, Class 2)	load_pct: 59.89 and 22.73
z_ij	The z-score (standardised value) of row i in column j	Row 1: (59.15 - 59.89) / 22.73 = -0.03

Target leakage: a feature built from the target gives a great score that cannot be repeated

	A	B	C	D
	load_pct	ambient_c	target_rounded	target
1	59.1	31.7	53	52.5
2	77.5	30.0	59	58.7
3	69.6	19.9	48	47.9

With that feature a max_depth 4 tree scores 0.54 kW. With honest features it scores 3.39 kW.

target_rounded is the target rounded to a whole number. At prediction time the target is unknown.

A feature recorded after the event is also target leakage.

Keep this checklist: five questions before you trust a score



Was the score measured on held-out data (rows the model never saw)?



Were the imputer and the scaler fitted on the training set only (no preprocessing leakage)?



Is every feature available at the moment you need the prediction, and none built from the target (no target leakage)?



Is the test set (group E6) still sealed, to be opened once after the final choice?



Is the score a cross-validation mean, shown with its standard deviation?

The same code can give different numbers, so we fix the random seed (random_state) and write it down

random_state (the random seed)	max_depth 10 tree: validation MAE	Shuffled 5-fold: mean MAE	max_depth chosen by the 5 percent rule
1	2.93	3.24	6
17	2.77	2.90	7
99	3.02	2.87	8

MAE in kW. Only the seed changed. Even the chosen max_depth moves: with 15 validation rows it is partly luck.

Write down what you ran, so anyone can repeat it

Run record

Data file
engineering_data.csv, 91 rows in the file

Split
Training E1 to E4, validation E5, test E6 sealed

Random seed
random_state = 17 (Step 5 tries 1, 99)

Versions
Python, NumPy, pandas, scikit-learn

Anyone with this record should get your numbers. After cleaning in Class 1, 88 rows remain: 58, 15, 15.

The notebook prints the record for you.

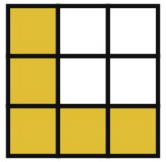
Paste it under every result you report.

Trust comes from four habits



Held-out data

Score on rows the model never saw



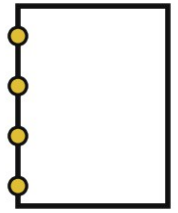
Cross-validation

Report mean and standard deviation



No leakage

Fit every preprocessing step on the training set



Reproducibility

Record seed, file, versions

Next class: features and choosing a model, using these four habits.

Read the workflow before you write it

```
pipeline = make_pipeline(imputer, tree_model)
group_kfold = GroupKFold(n_splits=4)
cv_scores = -cross_val_score(
    pipeline, X_train, y_train,
    groups=groups, cv=group_kfold,
    scoring="neg_mean_absolute_error")
print(cv_scores.mean(), cv_scores.std(ddof=1))
```

In plain words

make_pipeline chains the imputer and the tree into one model (a Pipeline).

GroupKFold(n_splits=4): each fold holds out one equipment group.

cross_val_score runs the pipeline on every fold: one score per fold.

The scoring gives the MAE with a minus sign; the minus in front turns it back.

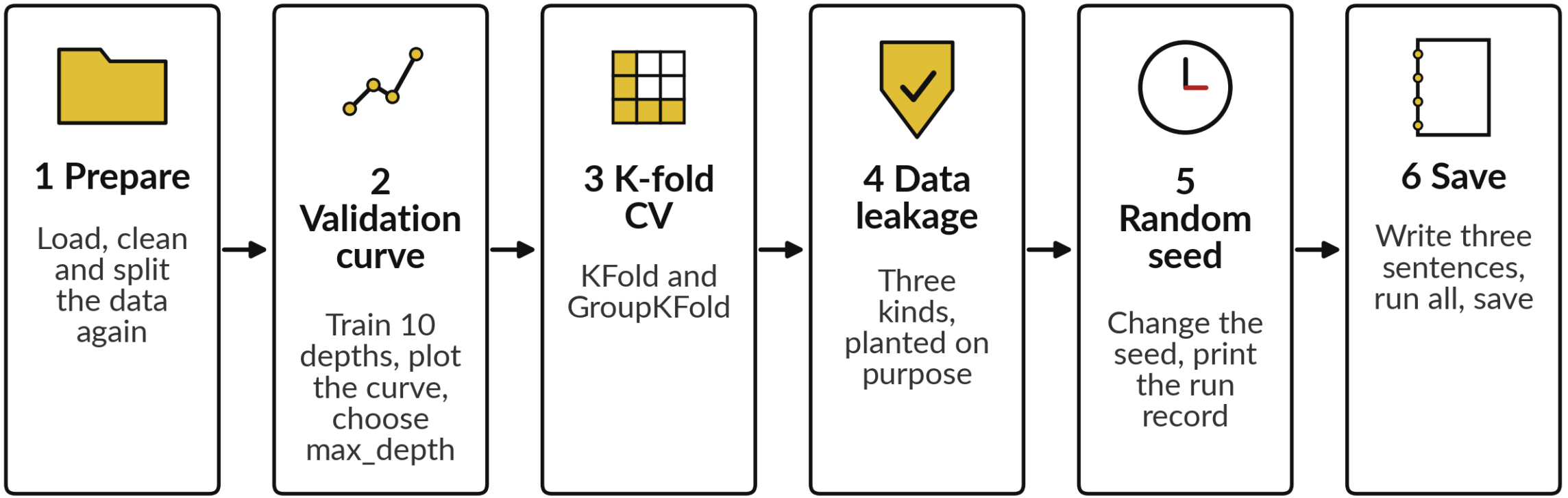
std(ddof=1): sample standard deviation (divide by k - 1).



Part 2: the lab

Close the slides. Open the notebook. Same steps, your own data.

The lab has six steps, and the first five end with a check



In each step, predict first, run second, then read the output. Each of Steps 1 to 5 ends with a CHECK of the numbers you should see.

Check your numbers against these before you move on

	What you print	You should see
	**Rows: training, validation, test**	58, 15, 15
	**Validation MAE at max_depth 1 and at 10**	5.30 and 2.77 kW
	**Chosen max_depth**	7
	**K-fold, k = 5: mean and standard deviation**	3.27 and 0.33
	**GroupKFold, k = 4: mean and standard deviation**	2.85 and 0.52
	**max_depth 10: honest and leaky validation MAE**	2.77 and 0.00
	**max_depth 10 tree, random_state 1 and 99**	2.93 and 3.02

MAE in kW. Standard deviations divide by k - 1 (ddof=1). If your number differs, stop and ask. With another scikit-learn version a difference in the second decimal is fine.

Four problems you will meet, what they mean and what to do

What you see	What it means	What to do
NameError: X_train	Step 1 was not run	Run all cells from the top, in order
An error that mentions groups	GroupKFold needs the group labels (a groups argument)	Pass groups=groups
All scores are negative	scikit-learn gives errors a minus sign (neg_ scoring: bigger is better)	Put a minus in front: -scores
Numbers differ a little from the slide	The random seed (random_state) is missing or different	Use random_state=17 in every tree, except in Step 5 where the seeds 1 and 99 are tested on purpose

How this class counts towards your coursework



Nothing is submitted today. Block 2 (Classes 3 and 4) is practice: it does not count toward your final mark, but you get feedback.



Its assignment has three parts: A9 (validation and trust, today's lab), then A10 (features) and A11 (model recommendation) from Class 4.



The Block 2 assignment closes on Tuesday 1 December, 11:00 (Kampala time).



The Block 2 quiz opens in the last 20 minutes of Class 4, stays open 24 hours, and covers both classes.

Save the notebook and keep the checklist

What you keep from today

Your saved lab notebook, the chosen max_depth, and the five-question checklist for trusting a score.

Our question today

How do we know a model that predicts cooling load will still work on a building it has never seen?

Next class

Class 4: features and choosing a model, plus the project brief.

Take away

Notebook
Restart, run all, save

max_depth
Chosen on the validation set

Checklist
Five questions

Next class
Features and models